FILE_GUIDE.md

Type: REF | Domain: AIStudio | Status: ACTIVE Version: 1.0.0 Created: 2026-05-20 | Last updated: 2026-05-20 | Owner: Manuel Barbero

How to use this guide

This is a **functional/relational** file guide — answers "what does each file do, what does it consume, what does it produce, what depends on it" rather than just "where does it live."

§1 The naming convention contract

All AIStudio files follow a consistent naming convention. Quick reference:

TYPE	Class	What this file IS	When you produce one	What reads it
STD	B	Codified standard. Authoritative on its scope.	Codifying a rule that's been validated.	Reference when in doubt.
REF	B	Reference material. Cheat sheets, FILE_GUIDE itself.	Producing a navigational/onramp doc.	Ad hoc lookup.
WIP	B or A	Work-in-progress. Plan, audit, draft.	Capturing a multi-session investigation.	Future sessions; promoted to NOTES or absorbed into STD.
NOTES	A or B	Session-bound or topical observations. Empirical record.	Capturing what was learned, not what was decided.	Cross-session continuity; promoted to STD if pattern emerges.

TYPE	Class	What this file IS	When you produce one	What reads it
CONCEPT	B	Foundational explanation. "Why this exists."	Defining a primitive or initiative.	Onboarding.
SESS	A	Permanent session archive.	End of session, summarizes outcomes.	Cross-session memory.
PIPELINE	D	Living work tracker. No date in filename.	Continuously updated each session.	Every session at start (planning pass).
TMPL	B	Reusable template (Word, PowerPoint, etc.).	Codifying a document format.	Document creation.
RES	B	Resume variant. JOB domain.	Tailoring resume for an application.	Application submission.

Filename pattern (universal): `TYPE - DOMAIN - TOPIC - DATE[ - HHMM][.ext]` **Domains**: General, AIStudio, JOB, urc

Class A (operational, multiple per day) → DATE + HHMM required **Class B** (standards, daily revision max) → DATE only, letter suffix for same-day **Class C** (code/SDLC) → no date in filename; version inside file **Class D** (living docs) → no date in filename; date in `*Last Updated*` field

---

§1a File Versioning Conventions

AIStudio uses two versioning patterns depending on file type. Both are mandatory for any file that is deployed or tracked.

Shell scripts (`.sh`)

Two lines, both required, **both must match**:

```
# Version: X.Y.Z
VERSION="X.Y.Z"
```

Python files (`.py`)

Header comment block at the absolute top of the file, before all imports:

```
# Version: X.Y.Z
# Changelog: X.Y.Z — <ticket> description of change.
#            Continuation lines indented 12 spaces (aligned with description start).
# Changelog: X.Y.Z-1 — previous change (newest first).
```

Rules: - `# Version:` must match the most recent `# Changelog:` entry — single source of truth. - Entries are prepended (newest first) — never append at the bottom. - One entry per version bump. Two tickets in one bump is fine. - The header block appears before `from __future__ import annotations` and all imports.

Class C files — version inside file, not in filename

Per §1, Class C (code/SDLC) files carry no date in the filename. The `# Version:` header is the only version signal.

---

§2 The runtime layer — `src/` + `front_end/` + `data/`

What runs to answer a query.

`src/local_llm_bot/app/` — Python application

File	Function	Reads	Produces
`api.py`	FastAPI HTTP layer	Corpus metadata, query payload, system prompt	JSON response with citations
`rag_core.py`	RAG pipeline orchestration	Query, corpus index	Embedded query → retrieved chunks → reranked → LLM context
`ingest/pipeline.py`	Document ingestion (per-file chunked/ embedded/ upserted)	Files in `data/corpora/<name>/uploads/`, corpus metadata seed	`index.jsonl`, `manifest.jsonl`, Qdrant chunks
`ingest/loaders.py`			

File	Function	Reads	Produces
	File-type-specific loaders	PDF/DOCX/HTML/MD files	Plain text + page metadata
`ingest/chunking.py`	Text chunking with overlap	Plain text	Chunks with stable IDs
`vectorstore/qdrant_store.py`	Vector store interface	Embeddings, metadata	Qdrant collections, similarity search
`ollama_client.py`	Ollama HTTP wrapper	Model name, prompt	Embedding vectors, LLM completions
`config.py`	Environment-driven config	OS env vars	Settings dict

`front_end/rag_studio.html` — UI

Single self-contained HTML file. Opens directly in browser, no server. Communicates with FastAPI backend at `localhost:8000`.

Provides: corpus selector, chat with citations, corpus CRUD (new/delete/rename), file upload + per-file inspect, metadata edit (description, search_guidance), help modal.

`data/corpora/<name>/` — Document corpora

Each corpus is a self-contained folder:

File	What it is	Who writes it
`uploads/`	Source documents (PDF, DOCX, etc.)	User via UI Add Files
`<name>_corpus_metadata.yaml`	Description, search_guidance, sources[], runtime stats	Seed at create-time, updated at ingest
`index.jsonl`	One row per chunk: ID, source path, page, text	Ingest pipeline
`manifest.jsonl`	Per-file ingest record: chunks count, last_ingested_at	Ingest pipeline
`trash/`	Soft-deleted files (recoverable)	UI delete

Demo corpus ships with the repo. **Help corpus** is built from `docs/` + key root markdown. **User corpora** are private (gitignored).

`prompts/system.txt` — System prompt

Loaded by `api.py` at query time. Per-corpus `search_guidance` from `<name>_corpus_metadata.yaml` is appended dynamically.

---

§3 The user-facing command surface

All commands available after running `./ais_install` from repo root.

Command	What it does	When you use it	Reads / Produces
`ais_start`	Starts FastAPI + Qdrant + Ollama; opens UI	Beginning of every session	— / running services
`ais_stop`	Stops all services cleanly	End of every session	— / —
`ais_log`	Tails the live backend log	Debugging; watching queries	service logs / stdout
`ais_bench`	Runs benchmark suite against a corpus	Validating retrieval quality	`benchmarks/<corpus>/<corpus>_questions.yaml` / benchmark report
`ais_download_sec_10k`	Downloads SEC 10-K filings from EDGAR	Setting up SEC corpus (optional)	EDGAR / `data/corpora/sec_10k/uploads/`
`ais_ingest_sec_10k`	Ingests SEC filings		uploads/ / index.jsonl + Qdrant

Command	What it does	When you use it	Reads / Produces
	into AIStudio	After downloading SEC	
`ais_help`	Shows command reference	Forgot a command	`ais_command_help.txt` / stdout
`ais_install`	Installs/ updates user commands	Fresh install; new command	`ais_user_commands_manifest.yaml` / shell aliases in `~/.zshrc`
`ais_create_shortcut`	Creates AIStudio Dock/ Desktop icon (macOS)	Optional post-install	/ `~/Desktop/AIStudio.app`

§5 The session lifecycle — what files flow when

During session

What's happening	Files you read	Files produced
Routine work	docs/, README, HOWTO	—
Producing a versioned doc	Existing version	New version with bumped front-matter version
Deploying files	File in `~/Downloads/`	File at canonical path

§9 The repo root layout

User-visible (public, ships with repo)

File / Dir	What it does	Relationship
`README.md`	Product overview	Entry point for new users
`QUICKSTART.md`	Install + first-run guide	Read after README
`HOWTO.md`	Day-to-day usage recipes	Read for "how do I X"
`TUTORIAL.md`	Guided walkthroughs	Read after QUICKSTART
`CONTRIBUTING.md`	Contribution guide	Read for repo participation
`LICENSE`	License terms	Repo convention
`Makefile`	`make install`, `make test-unit`, `make test`, `make lint`	Build orchestration
`pyproject.toml`	Python project config (ruff, pytest)	Tool config
`requirements.txt`	Python dependencies	`pip install -r`
`ais_install`	User command installer	Reads `ais_user_commands_manifest.yaml`
`ais_user_commands_manifest.yaml`	Auto-generated user command list	Produced by install

File / Dir	What it does	Relationship
`ais_*.sh` (user)	User commands (see §3)	Aliased by `ais_install`
`prompts/system.txt`	LLM system prompt	Loaded by `api.py`
`front_end/rag_studio.html`	UI	Opened in browser
`src/`	Application source (see §2)	Imported by api.py
`tests/`	Test suite	Run by `make test`
`docs/`	User documentation + help corpus sources	Ingested into help corpus
`benchmarks/`	Benchmark harness + question files + reports	Run by `ais_bench`
`data/`	Corpus data (demo + help tracked; user corpora gitignored)	Read by `rag_core.py`

---

§11 Cross-cutting workflows

Workflow B: Producing a tailored resume

1. Read the Resume Development standard (template system + Canonical Timeline)
2. Read closest existing resume variant for proof points reuse
3. Produce `RES - Manuel Barbero - <Firm> - <Role> - <date>[<letter>].docx`
4. Update master tracker

---

§12 The file-to-action map

Quick lookup: who reads what, who writes what, what feeds into what.

File pattern	Reads	Writes	Feeds into
`<corpus>_corpus_metadata.yaml`	`api.py` (system prompt), UI Edit	Ingest pipeline, UI Edit	LLM routing, search_guidance display
`prompts/system.txt`	`api.py`	Manual edit	LLM system instructions
`data/corpora/<name>/uploads/*`	Ingest pipeline	UI Add Files	Corpus chunks → Qdrant
`data/corpora/<name>/index.jsonl`	`rag_core.py` retrieval	Ingest pipeline	Query → ranked chunks

§15 Version history

Version	Date	Changes
1.0.0	2026-05-20	Initial version.

★★★ ★★★